Finance Backend Interview Guide

Purpose Of This Document

This guide helps me explain the project confidently in an interview.
It is written in a "line by line" style, but grouped by meaningful code blocks so I can speak clearly instead of reading every single statement.
The goal is to explain:

- what I built
- why I structured it this way
- how each layer works
- what tradeoffs I made
- how I would talk about it in an interview

One Minute Project Summary

I built a Python finance tracking backend that supports transaction CRUD, filtering, summaries, role-based access, validation, SQLite persistence, seeded demo data, and automated tests.
The system has three roles:

- ‘viewer‘ can view their own basic data
- ‘analyst‘ can use filters and advanced summaries
- ‘admin‘ can manage users and transactions globally

I intentionally kept the API thin and pushed business logic into a service layer, while SQL lives in repository classes.
This separation makes the code easier to read, test, and extend.

Best Interview Opening

If the interviewer says, "Walk me through your project," I should say:

"I built a finance backend in Python with SQLite. The main goal was to show clean backend design, not just CRUD. So I separated the code into API, service, repository, validation, permissions, database, and seed modules. The API handles routing and HTTP-style responses, the service layer handles business rules and role checks, and the repository layer handles SQL queries. I also stored money in integer cents to avoid floating-point issues, added summaries like income versus expense totals, and wrote tests for permissions, validation, pagination, and export."

Why I Chose This Design

1. Simple but strong stack

I used Python plus SQLite because the assignment values clean backend thinking more than framework complexity.
SQLite keeps the project easy to run locally and is enough to demonstrate persistence, filters, joins, and summary queries.

2. Separation of concerns

I did not put everything into one file.
Instead:

- ‘api.py‘ handles routing and request/response behavior
- ‘services.py‘ handles business rules
- ‘repositories.py‘ handles SQL
- ‘models.py‘ handles validation and normalization helpers
- ‘permissions.py‘ centralizes role checks
- ‘database.py‘ creates schema and connections

This makes the code easier to reason about in an interview.

3. Money safety

Amounts are stored as integer cents, not floats.
That avoids rounding bugs.
For example, ‘10.10 + 0.20‘ should stay exact in business systems.

4. Role-based behavior

The assignment asked for different user behaviors.
So I made the permissions meaningful:

- viewers cannot use advanced filters
- analysts can inspect richer data
- admins can mutate records and manage users

This shows business logic, not just endpoint creation.

How I Built The Project Step By Step

Step 1. Define the data model

I first identified the two main entities:

- ‘users‘
- ‘transactions‘

Each transaction belongs to a user and stores:

- amount
- type
- category
- date
- notes

This satisfies the core assignment requirement.

Step 2. Build the database schema

In ‘app/database.py‘, I wrote the SQLite schema and indexes.
I created:

- a ‘users‘ table
- a ‘transactions‘ table

- indexes on ‘user_id + entry_date‘, ‘entry_type‘, and ‘category‘

That supports filtering and analytics efficiently for a small assignment project.

Step 3. Add validation helpers

Before writing endpoints, I created normalization and validation helpers in ‘app/models.py‘.
This avoids duplicating validation logic in many routes.
Examples:

- parse positive integers
- validate roles
- validate entry types
- parse ISO dates
- limit text field sizes
- convert amount strings to integer cents

Step 4. Build repositories

After schema and validation, I wrote repositories for persistence.
Repositories are responsible for:

- creating users
- updating users
- creating transactions
- updating transactions
- deleting transactions
- querying filtered transactions
- generating overview and analytics queries

This keeps SQL out of the HTTP layer.

Step 5. Add the service layer

Then I built ‘app/services.py‘, which is the heart of the project.
It handles:

- authentication from the ‘X-User-Id‘ header
- role checks
- deciding which user can see which data
- validation orchestration
- pagination rules
- summary logic
- export behavior

This is where I show backend thinking in the interview.

Step 6. Add the API layer

Once business logic was stable, I connected it to routes in ‘app/api.py‘.
The API class:

- parses method, path, headers, and query params

- dispatches to service methods
- returns consistent JSON errors
- supports raw export responses for CSV and JSON files

Step 7. Seed data and entrypoint

I added ‘app/seed.py‘ so the evaluator can run the project instantly and test roles without manually creating users.
I added ‘run.py‘ as a very simple local server entrypoint.

Step 8. Tests

Finally, I wrote tests for:

- health endpoint
- authentication
- permission restrictions
- create, update, delete transaction flow
- analytics access
- validation failures
- pagination
- CSV export

That makes the project more credible in an interview.

Architecture Flow

When a request comes in, the flow is:

1. ‘run.py‘ starts the WSGI server.
1. The request reaches ‘FinanceAPI.handle‘.
1. The API authenticates the caller through ‘FinanceService.authenticate‘.
1. The route dispatches to a service method.
1. The service validates inputs and enforces permissions.
1. The service calls repository methods for database work.
1. Repository results return to the service.
1. The service serializes domain data into response-friendly dictionaries.
1. The API wraps the result in a consistent HTTP-style response.

File By File Implementation Walkthrough

‘run.py‘

Lines 10 to 16

This is the application entrypoint.

- Line 11 seeds the database so demo users and transactions exist.
- Line 12 builds the WSGI application.
- Lines 13 to 16 start the local server.

Why this matters:
I wanted the project to be runnable in one command.

That lowers setup friction for reviewers.

‘app/database.py‘

Lines 7 to 37

This block defines the schema.

- Lines 8 to 14 create the ‘users‘ table.
- Lines 16 to 27 create the ‘transactions‘ table.
- Line 26 creates the foreign key from transactions to users.
- Lines 29 to 36 add indexes for common query patterns.

Interview explanation:
"I used database-level constraints as a safety net. The application validates inputs, but the schema also enforces valid roles, valid entry types, and positive amounts."

Lines 40 to 46

‘connect‘ ensures the parent folder exists, creates a SQLite connection, enables dictionary-like rows, and turns on foreign key checks.

Why this matters:
It makes repository code cleaner because rows can be accessed by column name.

Lines 49 to 55

‘managed_connection‘ is a context manager that opens and closes connections safely.

Why this matters:
It keeps resource handling neat and prevents connection leaks.

Lines 58 to 61

‘initialize_database‘ runs the schema script and commits it.

Why this matters:
The schema is created automatically, so no separate migration step is needed for this assignment.

‘app/errors.py‘

Lines 4 to 11

‘AppError‘ is the base application exception.
It stores:

- ‘status_code‘
- ‘code‘
- ‘message‘
- optional ‘details‘

Why this matters:
Every error in the system can be converted into a consistent API response.

Lines 14 to 36

These subclasses map business failures to HTTP-style meanings:

- ‘ValidationError‘ -> 400
- ‘AuthenticationError‘ -> 401
- ‘PermissionDenied‘ -> 403
- ‘NotFoundError‘ -> 404
- ‘ConflictError‘ -> 409

Interview explanation:
"Instead of returning ad hoc error dictionaries everywhere, I normalized failure handling through custom exceptions."

‘app/permissions.py‘

Lines 6 to 9

‘require_roles‘ is a tiny helper, but it centralizes authorization checks.
If the user role is not allowed, it raises ‘PermissionDenied‘.

Why this matters:
It keeps role checking readable in the service layer.

‘app/models.py‘

Lines 8 to 10

These constants define the valid roles, valid entry types, and decimal precision policy.

Why this matters:
Magic strings are avoided and validation rules stay centralized.

Lines 13 to 14

‘utc_now_iso‘ generates timestamps in UTC and strips microseconds.

Why this matters:
The timestamps are consistent and API friendly.

Lines 17 to 27

‘parse_positive_int‘ validates query params or payload values that must be positive integers.
It is used for fields like:

- ‘user_id‘
- ‘page‘
- ‘page_size‘
- ‘limit‘
- ‘year‘

Why this matters:
I reuse one function instead of repeating similar validation logic across endpoints.

Lines 30 to 41

‘normalize_role‘ and ‘normalize_entry_type‘ turn input into normalized lowercase values and reject invalid options.

Why this matters:
The API becomes more predictable and easier to debug.

Lines 44 to 48

‘normalize_date‘ enforces ‘YYYY-MM-DD‘ input.

Why this matters:
Date handling is strict and summary queries can rely on ISO date ordering.

Lines 51 to 68

‘normalize_text‘ handles required and optional string fields, trims whitespace, and checks max length.

Examples:

- ‘name‘
- ‘category‘
- ‘notes‘

Lines 71 to 83

‘amount_to_cents‘ is an important business-safety function.

- It parses a decimal safely.
- It rejects invalid values.
- It rejects zero or negative amounts.
- It rejects more than two decimal places.
- It converts the final amount into integer cents.

Interview explanation:
"This was one of the most important validation choices because money should not be stored as floats."

Lines 86 to 87

‘cents_to_amount‘ converts database values back into two-decimal strings for API output.

Lines 90 to 109

‘validate_user_payload‘ validates the allowed user fields and supports both create and partial update.

Key idea:

- create requires all fields
- partial update only validates provided fields

Lines 112 to 139

‘validate_transaction_payload‘ does the same for transaction data.
It maps external API field names like ‘amount‘, ‘type‘, and ‘date‘ into internal names

like:

- ‘amount_cents‘
- ‘entry_type‘
- ‘entry_date‘

Why this matters:
The external API stays user friendly while the internal model stays explicit.

‘app/repositories.py‘

UserRepository lines 10 to 58

This class isolates user-related SQL.

- ‘list_users‘ returns all users for admin flows.
- ‘get_by_id‘ fetches a single user and raises ‘NotFoundError‘ if missing.
- ‘create_user‘ inserts a user and returns the created row.
- ‘update_user‘ merges partial updates and persists them.

Interview explanation:
"I return fresh database rows after write operations because it keeps the API response aligned with the stored state."

TransactionRepository lines 65 to 97

‘_base_query‘ and ‘_build_where_clause‘ help construct reusable SQL for transaction queries.

The supported filters are:

- ‘user_id‘
- ‘entry_type‘
- ‘category‘
- ‘date_from‘
- ‘date_to‘
- ‘search‘

Why this matters:
Instead of rewriting SQL for every endpoint, I made filtering composable.

Lines 98 to 125

‘list_transactions‘ returns paginated results plus a separate total count.

Important details:

- ordering is newest first
- page and page size are passed from the service layer
- count query supports pagination metadata

Lines 127 to 147

‘list_all_transactions‘ is used for exports.

Why this matters:

Exports should not be restricted by pagination.

Lines 149 to 171

‘get_by_id‘ loads a transaction with joined user information.
This is useful for display and access checks.

Lines 173 to 196

‘create_transaction‘ inserts a new row and returns the full created transaction.

Lines 198 to 227

‘update_transaction‘ merges the existing record with the validated update payload and writes the new values.

Why this matters:
Partial updates do not accidentally erase omitted fields.

Lines 229 to 234

‘delete_transaction‘ first verifies that the row exists, then deletes it.

Lines 236 to 249

‘overview‘ calculates:

- total transaction count
- total income
- total expenses

This supports the balance summary.

Lines 251 to 264

‘category_breakdown‘ groups by category and aggregates totals.

Lines 266 to 281

‘monthly_totals‘ groups by month for a chosen year and separately totals income and expenses.

Lines 283 to 301

‘recent_activity‘ returns the latest transactions using the same filter builder.

Interview explanation:
"The repository layer is where I concentrated SQL concerns like joins, grouping, ordering, and aggregation."

‘app/services.py‘

This is the most important file in the project because it contains the real application behavior.

Lines 22 to 27

The constructor initializes the database and wires up user and transaction repositories.

Lines 29 to 36

‘authenticate‘ reads ‘X-User-Id‘ from headers and loads the corresponding user.

Why this matters:
Authentication is intentionally simple for the assignment, but the role logic still becomes testable and realistic.

Lines 38 to 58

These user methods enforce admin-only operations for listing, creating, and updating users, while allowing a user to fetch their own profile.

Lines 60 to 72

‘list_transactions‘ handles pagination and passes validated filters to the repository.
It also returns pagination metadata.

Interview explanation:
"I treated pagination as part of the service contract, not as a controller detail."

Lines 74 to 112

‘export_transactions‘ supports two export formats:

- CSV
- JSON

The method:

- validates the format
- reuses the normal transaction filters
- loads all matching records
- serializes them
- builds either JSON bytes or CSV bytes

Why this matters:
It shows I can support multiple backend representations from the same underlying data model.

Lines 114 to 141

These transaction write methods are admin only.

- ‘get_transaction‘ first loads the record and checks visibility.
- ‘create_transaction‘ validates the payload and verifies the user exists.
- ‘update_transaction‘ supports partial updates.
- ‘delete_transaction‘ deletes and returns a confirmation payload.

Lines 143 to 153

‘get_overview‘ calculates:

- transaction count

- total income
- total expenses
- current balance

Balance is computed as income minus expense.

Lines 155 to 170

‘get_category_breakdown‘ is restricted to analyst and admin roles.
It optionally filters by transaction type and returns category totals.

Lines 172 to 188

‘get_monthly_totals‘ requires a valid year and builds month-by-month income, expense, and net values.

Lines 190 to 194

‘get_recent_activity‘ returns the latest transactions with a configurable limit.

Lines 196 to 217

The serializer methods convert internal database field names into cleaner API names.

Example:

- internal ‘entry_type‘ becomes API ‘type‘
- internal ‘entry_date‘ becomes API ‘date‘

Lines 219 to 223

‘_assert_transaction_visibility‘ ensures non-admin users can only access their own transactions.

Lines 225 to 239

‘_target_user_id‘ contains a subtle but important authorization rule.

- admins may request global data or specific users
- non-admin users are limited to their own user id

Why this matters:
This prevents horizontal privilege escalation.

Lines 241 to 253

‘_summary_filters‘ validates and applies summary date filters.
It also blocks advanced summary filters for restricted roles.

Lines 255 onward

‘_transaction_filters‘ handles transaction query rules.
The interesting part is that viewers can list their own transactions but cannot apply advanced filters like:

- category
- type

- date range
- text search

That makes the role design more meaningful.

‘app/api.py‘

Lines 17 to 29

‘ApiResponse‘ supports both JSON responses and raw file export responses.
This became necessary after I added CSV and JSON export.

Lines 32 to 57

‘FinanceAPI.handle‘ is the top-level request entry.

It does four things:

1. normalize the request method and headers
1. parse path and query string
1. handle public health checks
1. catch known and unknown exceptions and convert them into API responses

Why this matters:
It keeps error behavior consistent across the project.

Lines 59 to 71

‘wsgi_app‘ adapts the API class to the WSGI server.

Why this matters:
It lets the project run with the Python standard library server while still being structured like a real backend.

Lines 73 to 127

‘_dispatch‘ maps routes to service methods.

Key routes:

- ‘/health‘
- ‘/me‘
- ‘/users‘
- ‘/transactions‘
- ‘/transactions/export‘
- ‘/summaries/overview‘
- ‘/summaries/category-breakdown‘
- ‘/summaries/monthly-totals‘
- ‘/summaries/recent-activity‘

Interview explanation:
"I kept the routing layer intentionally thin. It knows how to decode the request and where to send it, but not how to make business decisions."

Lines 129 to 135

‘_json_body‘ ensures write requests receive valid JSON.

Lines 137 to 146

‘_headers_from_environ‘ converts WSGI environment values into normal request headers for the API class.

‘app/seed.py‘

Lines 8 to 20

This file defines default demo users and transactions.

Why this matters:
The reviewer can test all roles immediately.

Lines 23 to 34

‘ensure_seed_data‘ initializes the database and only inserts sample data if the users table is empty.

Why this matters:
Seeding is idempotent and safe to run on startup.

‘tests/test_api.py‘

This file proves that the main behaviors work.

Important tested flows:

- health endpoint is public
- authenticated user lookup works
- viewer permissions are enforced
- admin can create, update, and delete a transaction
- analyst can access advanced summary data
- invalid payloads return validation errors
- pagination works
- CSV export works

Best interview line:
"I used tests to prove the important backend rules, especially permissions and validation, because those are the easiest places for regressions."

Requirements Mapping

Financial records management

Covered by:

- create transaction
- list transactions
- get transaction
- update transaction
- delete transaction
- filtering by type, category, date range, and notes search

Summary and analytics logic

Covered by:

- overview totals
- current balance
- category breakdown
- monthly totals
- recent activity

User and role handling

Covered by:

- admin user management
- per-role access rules
- self-versus-global visibility rules

API/backend interface

Covered by:

- REST-style routes
- WSGI-compatible backend
- JSON and file export responses

Validation and error handling

Covered by:

- strict payload validation
- query param validation
- custom exception hierarchy
- consistent API errors

Database/persistence

Covered by:

- SQLite schema
- repository layer
- joins, filters, grouping, and indexes

Python code quality

Covered by:

- small focused modules
- reusable validation helpers
- service and repository separation
- tests and documentation

Technical Decisions I Should Mention In Interview

Why store money as cents?

Because floating-point arithmetic can produce precision problems.
Integer cents are safer for financial calculations.

Why use a service layer?

Because role logic, summary logic, and data visibility rules do not belong in raw SQL or routing.
The service layer is where business logic should live.

Why use repositories?

Because SQL should be centralized.
If I later move from SQLite to PostgreSQL or SQLAlchemy, the impact is isolated.

Why simplified authentication?

The assignment explicitly allowed simple assumptions.
So I used ‘X-User-Id‘ to keep focus on backend design and access logic.

Why seed data?

It improves reviewer experience and makes demos fast.

Interview Questions And Strong Answers

Q. How does a transaction creation request work end to end?

Answer:
"The API route parses the JSON body and forwards it to the service layer. The service validates the payload, checks that the caller is an admin, confirms the target user exists, and then calls the repository to insert the row. After insertion, the repository fetches the created record and the service serializes it into an API-friendly response."

Q. Where is authorization enforced?

Answer:
"Mainly in the service layer. I use a small ‘require_roles‘ helper for role checks, and I also use helper methods like ‘_target_user_id‘ and ‘_assert_transaction_visibility‘ to enforce ownership rules."

Q. How did you avoid repeating validation logic?

Answer:
"I centralized validation in ‘models.py‘ so both create and update flows reuse the same normalization functions for dates, money, roles, text length, and integer parsing."

Q. How would you scale this project?

Answer:
"I would swap the simple header authentication for JWT or session auth, move to PostgreSQL, add migrations, add structured logging, add more analytics endpoints, and introduce a framework like FastAPI for automatic OpenAPI docs. The current separation of layers makes that migration straightforward."

Q. What are the biggest strengths of this project?

Answer:
"The strongest parts are clean separation of concerns, solid validation, role-aware behavior, safe money handling, and meaningful analytics beyond basic CRUD."

Tradeoffs And Honest Limitations

I should be honest about these points:

- authentication is simplified
- no ORM was used
- no background jobs or async processing
- no production deployment config
- summary logic is sufficient for the assignment, not a full finance platform

This is good to say in an interview:

"I kept the project intentionally focused. I optimized for correctness, clarity, and maintainability instead of overengineering."

How I Should Close The Interview Discussion

If they ask, "What would you improve next?" I should say:

"Next I would add real authentication, migration tooling, OpenAPI documentation, stronger search and reporting, more tests around edge cases, and likely move to FastAPI plus PostgreSQL if the product scope grew."

Final Talking Point

If I want one polished closing line, I should say:

"The project demonstrates that I can translate a business brief into a clean backend design, model the data carefully, enforce permissions, validate inputs, write reusable Python code, and provide both CRUD and analytical behavior in a maintainable way."